Project : OS Command Injection Assessment

Name : Mohammad Aamir Siddique

Date : 03/03/2026

1. Project Objective

To demonstrate the identification, exploitation, and mitigation of OS Command Injection vulnerabilities in Linux-based web applications. This project focuses on simulating a real-world cybersecurity scenario to assess the security of a vulnerable application hosted on a Linux environment.

2. Scenario & Motivation

Many Linux-based web applications are vulnerable to OS Command Injection due to poor input validation. Such flaws can let attackers execute system commands, risking data breaches and server compromise. This project recreates a corporate environment to scan, exploit, and document these vulnerabilities, reinforcing ethical hacking skills and promoting secure coding practices.

3. Industry Tools Utilized

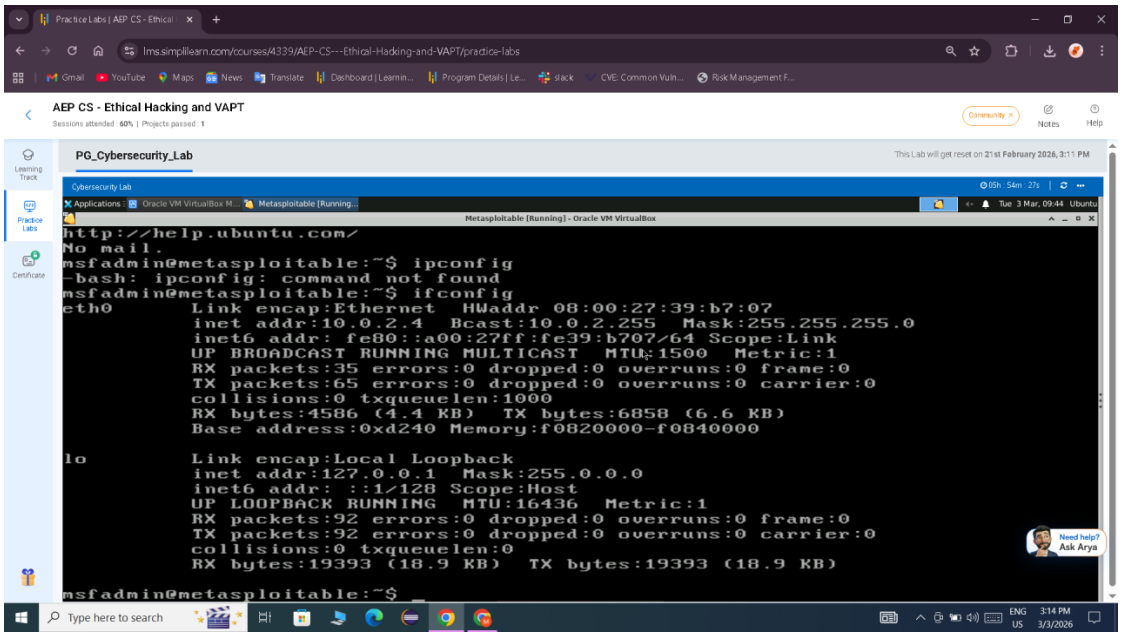
- **Kali Linux:** A preferred operating system for penetration testers and ethical hackers, pre-equipped with a wide range of cybersecurity tools.
- **Nikto:** An open-source web vulnerability scanner that helps security analysts detect server misconfigurations and potential entry points.
- **Metasploitable (DVWA):** A purposely vulnerable web application used by security teams to simulate real-world threats. *(Note: DVWA was utilized in the lab environment as the equivalent substitute for bWAPP).*

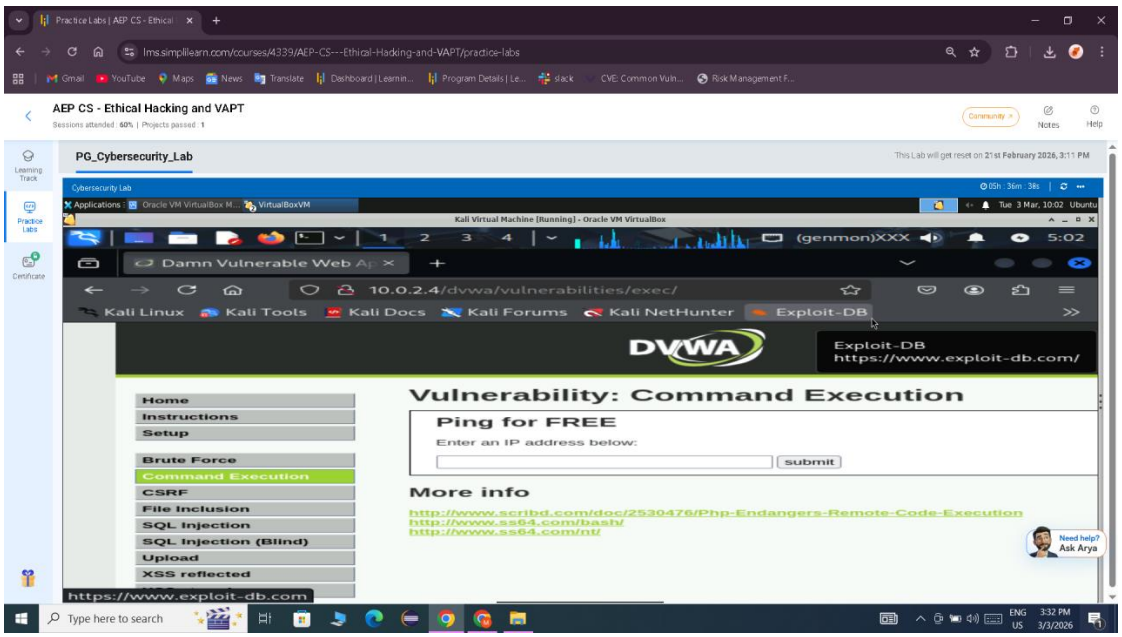
---

4. Execution Methodology & Forensic Log

Phase 1: Lab Environment Setup & Reconnaissance

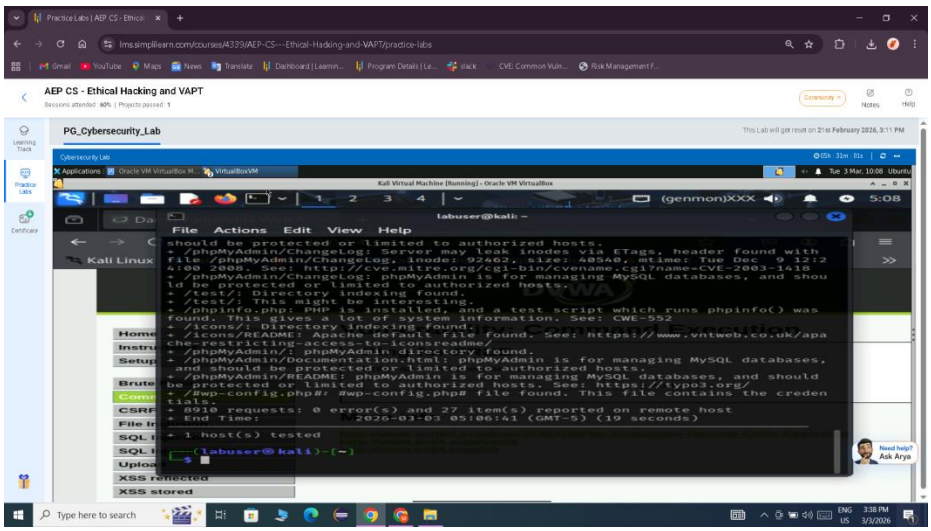
Description: The lab environment was established by deploying both the Kali Linux (attacker) and Metasploitable (target) virtual machines on the same local network. Initial reconnaissance was performed using the `ifconfig` command on the target server to identify its IP address (`10.0.2.4`) to facilitate the attack. The vulnerable DVWA web application was then successfully accessed via the Kali browser.





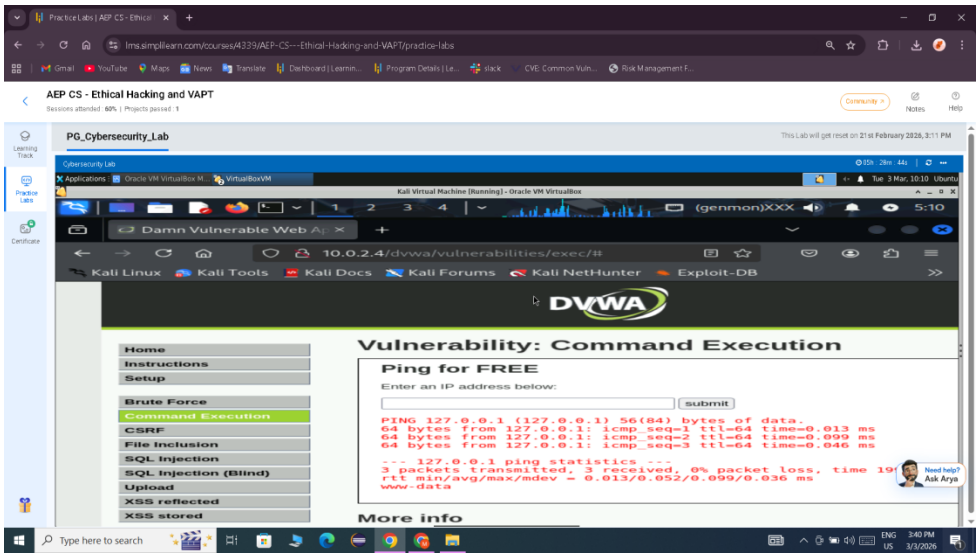
Phase 2: Vulnerability Scanning

Description: Before attempting direct exploitation, a surface-level scan was conducted to identify misconfigurations. The `nikto` scanner was executed from the Kali Linux terminal against the target IP, successfully revealing outdated software and exposed directories (such as `/phpMyAdmin/`).



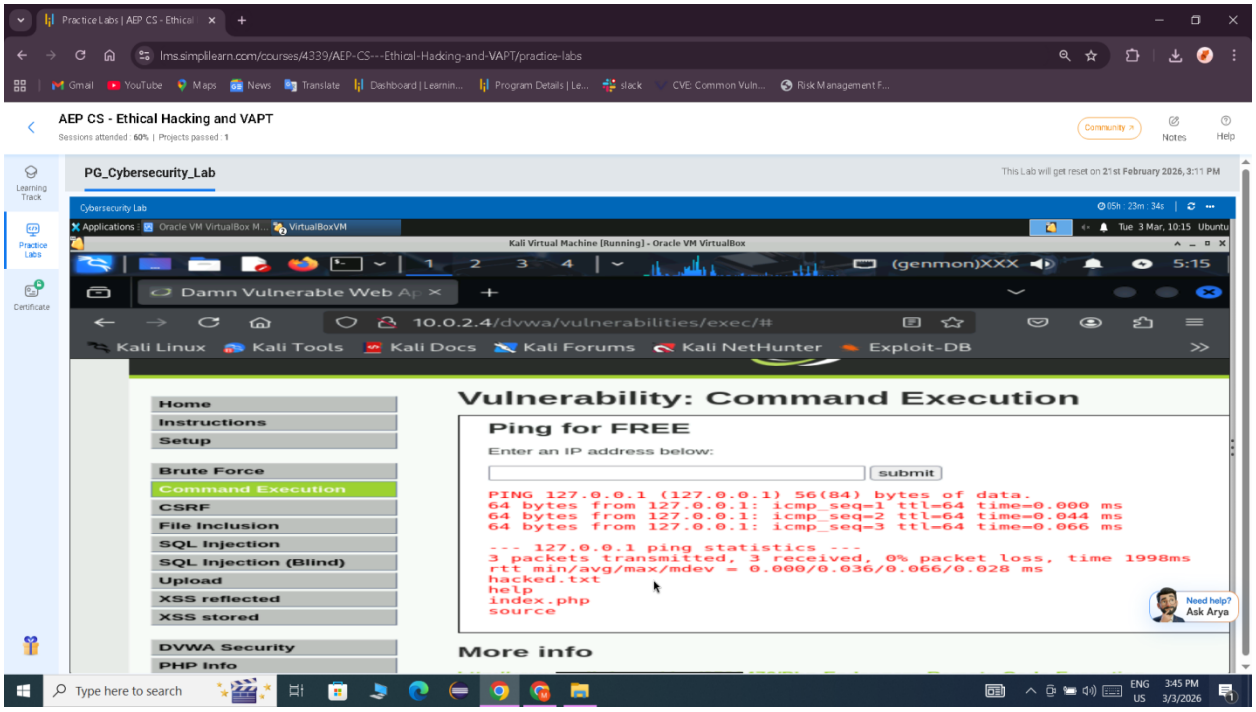
Phase 3: Direct OS Command Injection

Description: The DVWA "Command Execution" module was targeted to test for poor input validation. A malicious payload (`127.0.0.1 ; whoami`) was injected using the `;` operator. The application failed to sanitize the input, executing the ping command followed immediately by the system command. The server returned the output `www-data`, confirming unauthorized direct command execution.



Phase 4: Blind Command Injection & Operator Testing

Description: To demonstrate a blind command injection, a payload was injected to silently create a file on the server without relying on visual output (`127.0.0.1 ; touch hacked.txt`). To verify the success of this blind attack, a secondary payload utilizing the `&&` operator was deployed (`127.0.0.1 && ls`). The server successfully listed the active directory, revealing the newly created `hacked.txt` file, thereby proving complete remote code execution capabilities.



5. Conclusion & Mitigation

The assessment successfully demonstrated that the application is highly vulnerable to OS Command Injection due to an absence of input sanitization. To mitigate this OWASP Top 10 vulnerability, the development team must implement strict input validation, utilize parameterized functions rather than direct shell execution, and apply comprehensive system hardening practices.